

Structer, definition

Vi vill gärna använda vårt objektorienterade synsätt så långt möjligt, även när vi löser problem i C.

Antag t.ex. att vi ska göra något personregister. Med de konstruktionerna i C som vi känt till fram till nu skulle vi då typiskt hitta en lösning med en array för namnen en annan för telefonnumren, ytterligare en för skonummer etc. Detta är givetvis inte bra. Om vi till ex. gör sorteringsfunktioner så att man t.ex. kan sortera om sina vänner på basis av skostorleken, så skulle man under sorteringen få lov att plocka om elementen i de andra arrayerna också. En sådan implementation skulle bli väldigt kladdig och orobust. Dessutom så är det väldigt intuitivt att en person identifieras av {namn, telefonnummer, skonummer}, och vi har en lista av sådana personer.

Motiveringen för en mekanism som håller ihop olika typer av enklare data-komponenter till ett sammansatt komplext dataobjekt bör därmed vara klar. Om vi identifierar objekt i ett problem och klurar ut dessas egenskaper så finns möjligheten lagra data på detta sätt i C.

Mekanismen kallas **struct**. Formen påminner lite om klassdefinitioner i Java men är betydligt enklare:

```
struct structnamn {  
    "lista av komponenter"  
};
```

- **struct** är ett nyckelord
- structnamn är valfritt namn som vi döper structen (datatypen) till
- "lista av komponenter" är ett valfritt antal specifikationer på vilka datakomponenter som ska ingå i vår nya datatyp. Varje sådan har exakt samma form som en godtycklig variabeldeklaration, observera dock att det *inte är* en variabel deklARATION - det hela är bara en "mall" som beskriver hur en ny datatyp ser ut.

Exempel på en definition av en struct som beskriver en person:

```
struct t_person {  
    char name[30];  
    char phone[30];  
    int shoenummer;  
};
```

Definitionens naturliga plats är upptill i en C-fil. Om nödvändigt kan den delas mellan flera C-filer genom att lägga den i en av C-filernas h-fil. Det är dock bra om man kan undvika detta, då h-filen används till att "exportera" funktioner, och hela structen blir därmed också exporterad (trots att den kanske bara behöver delas med t.ex. en annan C-fil)

Structer, instansiering

För att skapa ett dataobjekt som är t.ex. en person enligt exemplet ovan, gör man en vanlig variabeldeklaration där hela **struct t_person** är typen, dvs:

```
struct t_person person;
```

deklarerar en variabel **person** som är av *typen* **struct t_person**. Och

```
struct t_person personlista[1000];
```

deklarerar en variabel **personlista** som är en array av *typen* **struct t_person**. Och

```
struct t_person *pp;
```

deklarerar en variabel **pp** som är av *typen* pekare till en **struct t_person** (dvs struct t_person*) etc.

Structer, access till instanser

Givet deklarationerna i föregående stycke

```
scanf(" %d", &person.shoenum);
```

är en inläsning från tangentbordet till komponenten **shoenum** i variabeln **person**

```
strcpy(personlist[42].name, "Allan");
```

kopierar stängen "Allan" till komponenten **name** i variabeln **personlist**[42]

```
personlist[0].phone[0] = 0;
```

"nollställer strängen" i komponenten **phone** i variabeln **personlist**[0]

```
pp = (struct t_person*)malloc(sizeof(struct t_person) * 100);
```

Allokerar minne för 100 dataobjekt av typen **struct t_person** och sätter pp att peka på det.

```
*pp = person;
```

Tilldelar det första dataobjektet som pp pekar på värdet av variabeln **person** (det sker alltså en kopiering av person till det som pp pekar på).

```
pp[42] = person;
```

Tilldelar det 43:a dataobjektet i det minne som pp pekar på värdet av variabeln **person** (det sker alltså en kopiering av det data som finns i **person** till det som pp+42 pekar på).

Slutsats: vi deklarerar och allokerar variabler, pekare, arrayer etc som har någon egendefinierad typ (dvs struct) på samma sätt som då vi deklarerar motsvarande objekt av någon bas typ (int, float etc). Betydelsen är också helt analog, dvs det allokeras plats för dataobjekt på ett sådant sätt som vi anger resp. att vi sätter ett namn på detta.

För att komma åt en enskild komponent i ett structobjekt används en punkt.

Vidare så kan vi referera till ett helt dataobjekt av en struct genom att ange ett variabelnamn som är av den structtypen, genom att indexera en array som är av structtypen eller genom att avreferera eller indexera en pekare som är av structtypen. Allt detta är helt analogt med hur bas typerna hanteras i språket.

Vi kan dock *inte* t.ex. addera två stycken personer. Med lite eftertanke så är detta självklart: hur skulle kompilatorn kunna ge någon semantik till addition. Även om vi definierar någon struct där man kan tänka sig att addition skulle ha en mening (t.ex. struct t_complex för komplexa tal), så kan ju inte kompilatorn veta hur vi vill definiera addition för komplexa tal. Och så har vi inte överlagring heller (dvs vi kan inte utöka operatorers förmåga så att de förstår ytterligare typer). Allmänt gäller att den enda operator som vi kan applicera på dataobjekt av structtyp är tilldelningsoperatoren, och den

kräver då två lika structobjekt.

Structer, mera...

Notera nu att structdefinitionen har en rekursiv karaktär: Komponentlistan i definitionen är en lista där varje komponent ser ut som vilken variabeldeklaration som helst. Och variabeldeklarationen kan använda sig av structdefinitionen. En struct kan alltså innehålla en struct. För att det inte ska uppstå objekt av oändlig storlek genom att en struct innehåller sig själv, så måste just detta förbjudas. Däremot går det bra med någon annan struct. Det går också bra med pekare till objekt av samma typ. Ex:

```
struct t_person {  
    char name[30];  
    char phone[30];  
    int shoenummer;  
    struct t_person *pal;  
};
```

för att hålla ihop kompisarna.
Eller struct i struct:

```
struct t_address {  
    char street[30];  
    char postaladress[30];  
};  
struct t_person {  
    char name[30];  
    char phone[30];  
    int shoenummer;  
    struct t_address address;  
};
```

för att generalisera adressen.
Exempel på hur structobjektens komponenter accessas då vi slår ihop de två exemplen ovan:

```
struct t_address {  
    char street[30];  
    char postaladress[30];  
};  
struct t_person {  
    char name[30];
```

```

char phone[30];
int shoenummer;
struct t_address address;
struct t_person *pal;
};
...
struct t_person person;
...
printf("%s är kompis med %s", person.name, (*person.pal).name);
printf("%s bor på %s och har telefonnummer %s", person.name, person.address.street, person.phone);
printf("%s initial är %c", person.name, person.name[0]);

```

Det är alltid namet längst till vänster som anger vilket dataobjekt (variabel etc.) som avses, t.ex. är det den deklarerade variabeln `person' som avses i alla fallen i exemplet ovan, dvs person.name, person.address.street, person.phone, (*person.pal).name

Om vi anser att den biten därmed är avklarad så blir det sen enklare om vi försöker förstå typerna. `person' har typen struct person (framgår av deklarationen). Då innehåller den bland annat komponenten `name' den får vi fram genom att lägga till .name. Vi har då något som är av den typ som name har, vilket vi finner är char-array, eller mer formellt char *. Hela **person.name** har alltså typen char*. Om vi vill komma åt ett specifikt element i den arrayen så är det bara att indexera hela (det har ju typen char* så det verkar naturligt), person.name[0] har alltså typen char.

person är struct t_person, person.address är struct t_address och person.address.street är char* (det är bara att börja med variabeln som står först, gå till deklarationen och finna typen, gå till dess definition för att hitta nästa etc.)

En till enligt samma metod: person är struct t_person, person.pal är struct t_person*, *person.pal är struct t_person och slutligen (*person.pal).name är char*. (Parenteserna är nödvändiga i detta fall pga operatorprioriteter)

Structer, typedef

För att öka bekvämligheten så finns det möjlighet att göra typdefinitioner i C, detta har inget med structer att göra, men det används väldigt ofta till just structer och sällan till annat, därför får det komma in här. Ex:

```
typedef struct t_person t_person;
```

Här definierades t_person (det kursiverade) till att bli av typen struct t_person. Vi kan nu använda t_person istället för det lite klumpiga struct t_person om vi vill. Exempel:

```

typedef struct t_person t_person;
struct t_person {
    char name[30];
    char phone[30];
    int shoenummer;

```

```

    t_person *pal;
};
...
t_person person, *pp;
...
```

Man kan även baka ihop allt i ett, så här:

```

typedef struct t_person {
    char name[30];
    char phone[30];
    int shoenummer;
    struct t_person *pal;
} t_person;
```

En begränsning är att då kompilatorn träffar på pekaren `pal` så är typ-definitionen ännu inte klar. Därför måste man till denna komponent använda varianten med **struct t_person**.

Funktioner och structer

Här finns egentligen inte så mycket nytt. Om vi accepterar att en structdefinition, vare sig det finns en **typedef** eller ej, skapar en *ny typ*, så är hanteringen helt analog med hanteringen av bas typerna, dvs då man sätter returtypen eller typerna på parametrarna i en funktionsdefinition (eller prototypdeklaration) så kan de som bekant sättas till någon känd typ. Detta inkluderar förså de egendefinierade typerna.Ex:

```

typedef struct t_person {
    char name[30];
    char phone[30];
    int shoenummer;
    struct t_person *pal;
} t_person;
...
t_person *make_person(void);
t_person get_person_data(void);
void print_person_data(t_person person);
void update_person_data(t_person *person);
```

I ordning: `make\_person` måste allokera minne dynamiskt för att vara meningsfull. En pekare till detta returneras. `get\_person\_data` kan t.ex. tänkas ta in persondata från tangentbordet och returnerar *ett helt* structobjekt (ej dynamiskt minne) som innehåller dessa data. `print\_person\_data` kan t.ex. tänkas skriva ut data om en person i ett terminalfönster, och anroparen får passa *ett helt* structobjekt på stacken. `update\_person\_data` har möjligheten att både

läsa och ändra värden i det structobject som anroparen passar över en pekare till. Exempel på implementationer:

```
t_person *make_person(void)
{
    return (t_person*)calloc(sizeof(t_person), 1);
}
t_person get_person_data(void)
{
    t_person p;
    scanf("%s %d", p.name, &p.shoenum);
    return p;
}
void print_person_data(t_person person)
{
    printf("%s har tel. %s", person.name, person.phone);
}
void update_person_data(t_person *person)
{
    (*person).shoenum++;
    printf("%s fötter har växt till %d (gäller i resten av programmet)", (*person).name, (*person).shoenum);
}
```

Man kan alltså i vissa situationer välja bland alternativa sätt. Man kan säga att den första och sista metoden ger funktionen större möjligheter. De tar också mindre plats på stacken och är effektivare. Därför är det också de vanligaste sätten.

Skriv själv anropen till funktionerna ovan givet variabeldeklarationen

t_person person, *pp; Om du är osäker om det är rätt så hacka in koden och testa, eller maila mig.

Piloperatorn

Piloperatorn är inget djupt. Bara ett alternativt skrivsätt. Eftersom det är så vanligt med pekare till structobjekt (länkade listor, träd, "överordnade" structer etc) och formen för att accessa nextpekaren etc. alltid blir (*p).next så har man infört p->next som alternativt skrivsätt för detta (alltså minus-tecken och större än-tecken).

Operationer på bitar

Man kan ibland vilja operera på enskilda bitar i ett heltal, speciellt vid maskinnära programmering. Om man vill att a ska få det värde som bildas då bit 5 och 6 är "satta" (dvs båda är 1) och alla andra bitar är 0, så skriver man

```
a = 3 << 5;
```

dvs talet 11 (binärt) skiftat 5 bitar åt vänster och a får värdet 96 (decimalt).

Nytt exempel: Om vi vill att bit 8 ska bli 0 i variabeln a men resten bibehållas så blir det

```
a &= ~(1 << 8);
```

Först fixar vi fram en etta på bit 8, sedan tar vi komplementet till detta (dvs alla bitar byter tillstånd, så då är den en nolla i bit 8 och resten är ettor.

Slutligen gör &= (dvs $a = a \& \sim(1 \ll 8)$) dvs bitvis AND på de två heltalen och resultatet läggs i a. Just denna operation kallas oftast "maskning".

Nytt exempel Om vi vill sätta bit 4 i a och lämna resten av bitarna orörda så blir det:

```
a |= 1 << 4;
```

vilket kan uttryckas på binär form (symboliskt detta är inte C-kod) $a = a | 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001\ 0000$, där | betyder bitvis OR.

Man kunde lika gärna ha skrivit `a |= 16;`

Eftersom vi vet att $1 \ll 4$ blir värdet 16, men genom att använda skift så ser vi tydligare vad som avses, nämligen bit 4.

Om du vill kan du läsa själv om bitfält. Det är ett lite bekvämare sätt att arbeta med bitar som dock till skillnad från ovanstående inte är protabelt.